National University of Computer & Emerging Sciences, Karachi
Computer Science Department
Spring 2026, Lab Manual – 10

Course Code: CL-1004	**Course : Object Oriented Programming Lab**
Instructor :	**Muhammad Owais**

LAB - 10

Abstract Classes & Nested Classes

Abstract Class:

A class which is declared as abstract is known as an **abstract class**. It can have abstract and non-abstract methods. It needs to be extended and its method implemented. It cannot be instantiated.

- An abstract class must be declared with an abstract keyword.
- It can have abstract and non-abstract methods.
- It cannot be instantiated.
- It can have constructors and static methods also.
- It can have final methods which will force the subclass not to change the body of the method.

Syntax

```
abstract class A{}
```

Abstract Method:

A method which is declared as abstract and does not have implementation is known as an **abstract method**.

Syntax

abstract *void myFunction( ); //no method body and abstract*

Example

```
abstract public class Vehicle {
    abstract public void run(); // abstract method

}
```

It is the responsibility of child class(es) to override the abstract function and "complete" the parent class.

Example

```
class Car extends Vehicle{
    public void run(){
        System.out.println("Running Fast");
    }
}
```

Example: **(***Using abstract class from main)*

```
public class Main
{
    public static void main(String[] args) {
        Vehicle obj = new Car();
        obj.run();
    }
}
```

Output:

```
Running Fast


...Program finished with exit code 0
Press ENTER to exit console.
```

Point to Remember: We cannot create an instance of the abstract class.

```
abstract  class Animal {
    abstract public void eat(); // abstract method

}

public class Main
{
    public static void main(String[] args) {
        Animal obj = new Animal(); // trying to creating the insance
                                   //of abstract class
        obj.eat();
    }
}
```

Compilation failed due to following error(s).

```
Main.java:10: error: Animal is abstract; cannot be instantiated
            Animal obj = new Animal();
                         ^
```

Point to Remember: An abstract class can contain concrete functions.

Example:

```
abstract  class Animal {
    abstract public void eat(); // abstract method
    public void mov(){
        System.out.print("Animal is moving"); // concrtete method
    }

}
```

```
public class Main
{
    public static void main(String[] args) {
        Animal obj = new Dog();
        obj.mov();
    }
}
```

```
Animal is moving
```

Note: If there is an abstract method in a class, that class must be abstract.

Example:

```java
class Animal {
    abstract public void eat(); // abstract method
}
class Dog extends Animal{
    public void eat(){
        System.out.println("Animal is Eating ");
    }
}
public class Main
{
    public static void main(String[] args) {
        Animal obj = new Dog();
        obj.eat();

    }
}
```

```
Main.java:1: error: Animal is not abstract and does not override abstract method eat() in Animal
class Animal {
^
1 error
```

Anonymous Class:

If we do not have a child class, we can still override the abstract function by creating an **Anonymous Class.**

- A new class is defined (without a name, so called anonymous class)
- This new class extends abstract base class
- Abstract methods are overriden in this new class
- New instance of this new class is created and assigned to the parent variable

Example

```java
abstract  class Animal {
    abstract public void eat(); // abstract method
}

public class Main
{
    public static void main(String[] args) {
        Animal obj = new Animal() {
            public void eat(){
                System.out.println("Animal is eating...");
            }
        }; // end of Anonymous class
        obj.eat();

    }
}
```

```
Animal is eating...
```

Nested Classes

Java inner class or nested class is a class that is declared inside the class or interface.

It can access all the members of the outer class, including private data members and methods.

Syntax of Inner class

```
class Java_Outer_class{
 //code
 class Java_Inner_class{
  //code
 }
}
```

Need of Java Inner class

Sometimes users need to program a class in such a way so that no other class can access it. Therefore, it would be better if you include it within other classes.

If all the class objects are a part of the outer object then it is easier to nest that class inside the outer class. That way all the outer class can access all the objects of the inner class.

Types of Nested classes

There are two types of nested classes non-static and static nested classes. The non-static nested classes are also known as inner classes.

- Member inner class
- Anonymous inner class
- Local inner class

Java Member Inner class

A class that is created inside a class but outside a method is called member inner class. It is also known as a regular inner class. It can be declared with access modifiers like public, default, private, and protected.

Example:

In this example, we are creating a msg() method in the member inner class that is accessing the private data member of the outer class

```
class JavaOuterClass
{
  // private variable of the outer class
  private int value = 30;
  // inner class
  class JavaInnerClass
  {
    // public variable of the inner class
    public int getValue()
    {
      System.out.println("This is the getValue method of the inner class:");
      return value;
    }
  } //inner class end here
  public static void main(String args[])
  {
    //Creating object of outer class
    JavaOuterClass outer = new JavaOuterClass();
    // Creating object of inner class
    JavaOuterClass.JavaInnerClass inner = outer.new JavaInnerClass();
    System.out.println("Value:" + inner.getValue());
  }
}
```

Output:

```
This is the getValue method of the inner class:
Value:30
```

How to instantiate Member Inner class in Java?

An object or instance of a member's inner class always exists within an object of its outer class. The new operator is used to create the object of member inner class with slightly different syntax.The general form of syntax to create an object of the member inner class is as follows:

Syntax:

OuterClassReference.new MemberInnerClassConstructor();

Example:

obj.new Inner();

Here, OuterClassReference is the reference of the outer class followed by a dot which is followed by the new operator.

Java Local inner class

A local inner class in Java is a class defined inside a method or a block (like a for-loop or if-statement). It is not a member of the outer class, so it cannot have access modifiers (like public or private), but it can be final or abstract. A local inner class can access fields of the enclosing class, and to use its methods, you must create an object of it inside the method where it is defined.

Java local inner class example

```
public class localInner1{
 private int data=30;//instance variable
 void display(){
  class Local{
   void msg(){System.out.println(data);}
  }
  Local l=new Local();
  l.msg();
 }
 public static void main(String args[]){
  localInner1 obj=new localInner1();
  obj.display();
 }
}
```

Output

```
30
```

Anonymous class

An anonymous class in Java is a class without a name that is defined and instantiated at the same time, usually to override methods of an existing class or implement an interface for one-time use.

Java local Anonymous class example:

```
abstract class Greeting {
    abstract void sayHello();
}

public class Main {
    public static void main(String[] args) {
        // Anonymous class implementing Greeting
        Greeting g = new Greeting() {
            public void sayHello() {
                System.out.println("Hello from anonymous class!");
            }
        };

        g.sayHello(); // Output: Hello from anonymous class!
    }
}
```

Output:

```
Hello from anonymous class!
```